

VLDB 2026 Conference Keynote 2: Efficient and Reliable Systems for Building Foundation Models at Scale

Jingren Zhou, Alibaba Cloud

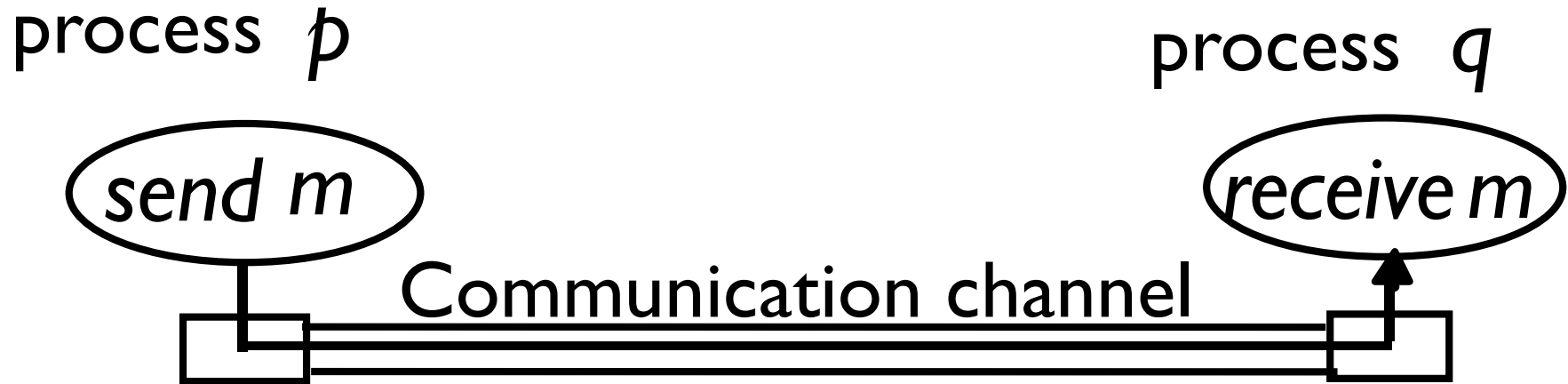


The rapid evolution of AI has led to increasingly large and complex foundation models with trillions of parameters and heterogeneous architectures, making efficient and reliable training and inference systems essential. Training such models requires large-scale distributed computation, effective overlap of computation and communication, sophisticated parallelization strategies, and robust fault-tolerant mechanisms. In addition, the rise of multi-turn and autonomous agents drives new demands for asynchronous reinforcement learning and post-training systems. Inference systems, on the other hand, must support diverse workloads with varying service-level agreements (SLAs), rapidly integrate engineering optimizations, and carefully balance trade-offs among throughput, latency, cost, and availability, particularly in distributed environments. In this talk, I will discuss the major systems challenges involved in building large-scale foundation models, drawing on our experiences developing Qwen large language models and Vwan video generation models. I will also present ongoing research and system co-designs for improving training, post-training, and inference at scale, with the goal of making complex AI workloads in cloud environments more efficient, agile, and reliable.

Jingren Zhou is Chief Scientist and Senior Vice President at Alibaba

Basics of Inter-process Communication

Inter-Process Communication



Inter-Process Communication



- ❑ One communication “unit” consists of two primitives
 - ☆ The **send** primitive is called by the sending process (caller, sender)
 - ☆ A corresponding **receive** primitive must be called by the receiving process (callee, receiver)

As programs

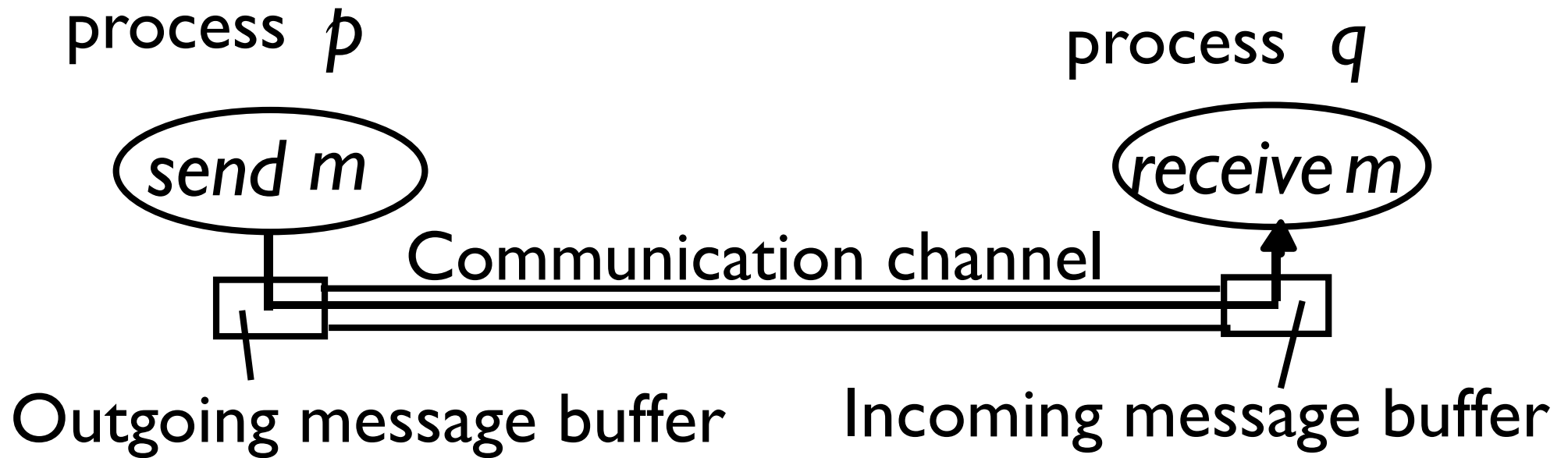
□ Program 1 running in process p

```
string m = "Hello World";  
...  
send (q, m);  
...  
int x = 1+1;  
m = "something else";
```

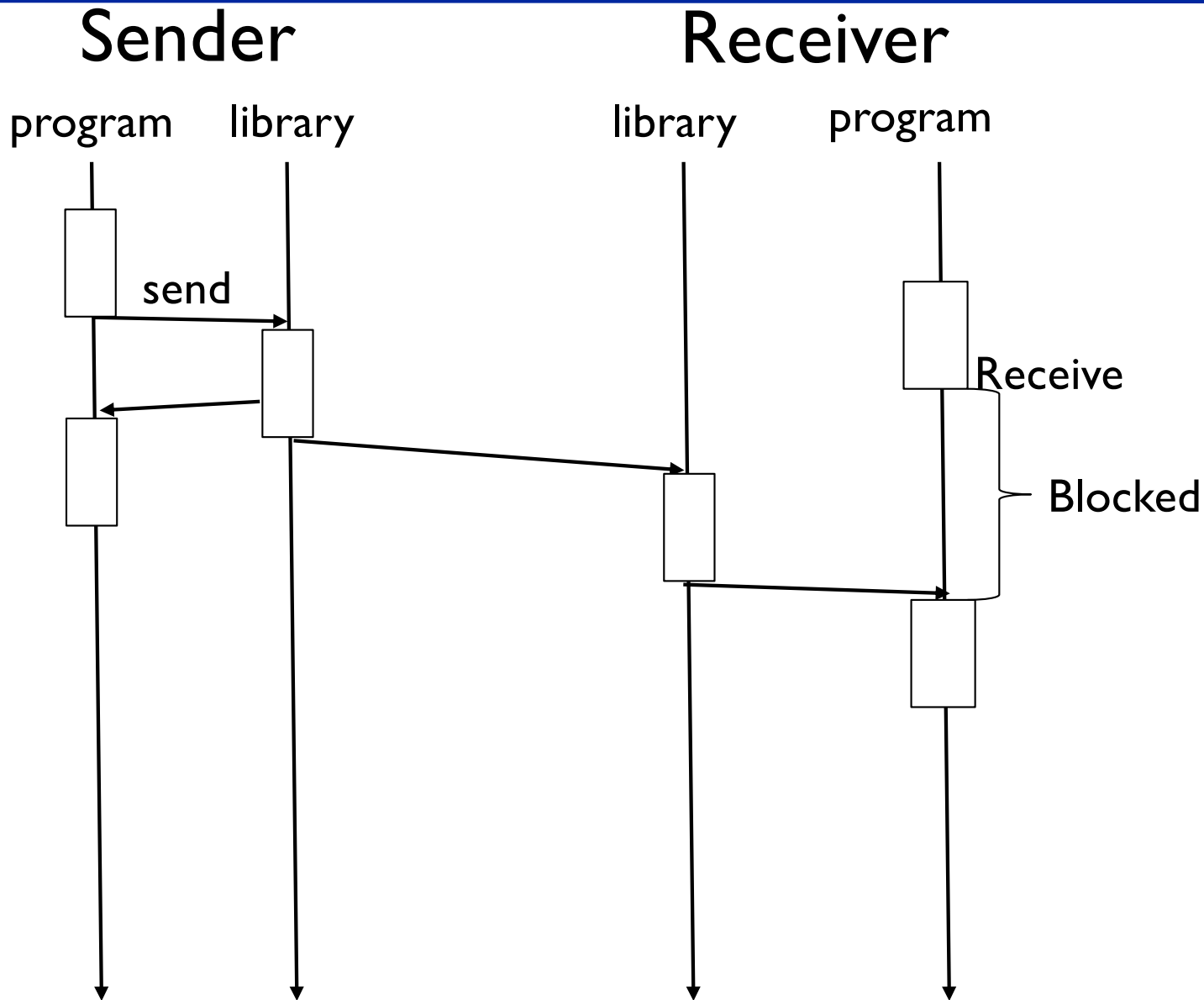
□ Program 2 running in process q

```
string m;  
char s;  
  
(m, s) = receive();  
print ("received: ");  
print m;  
print (" from: ");  
print s;
```

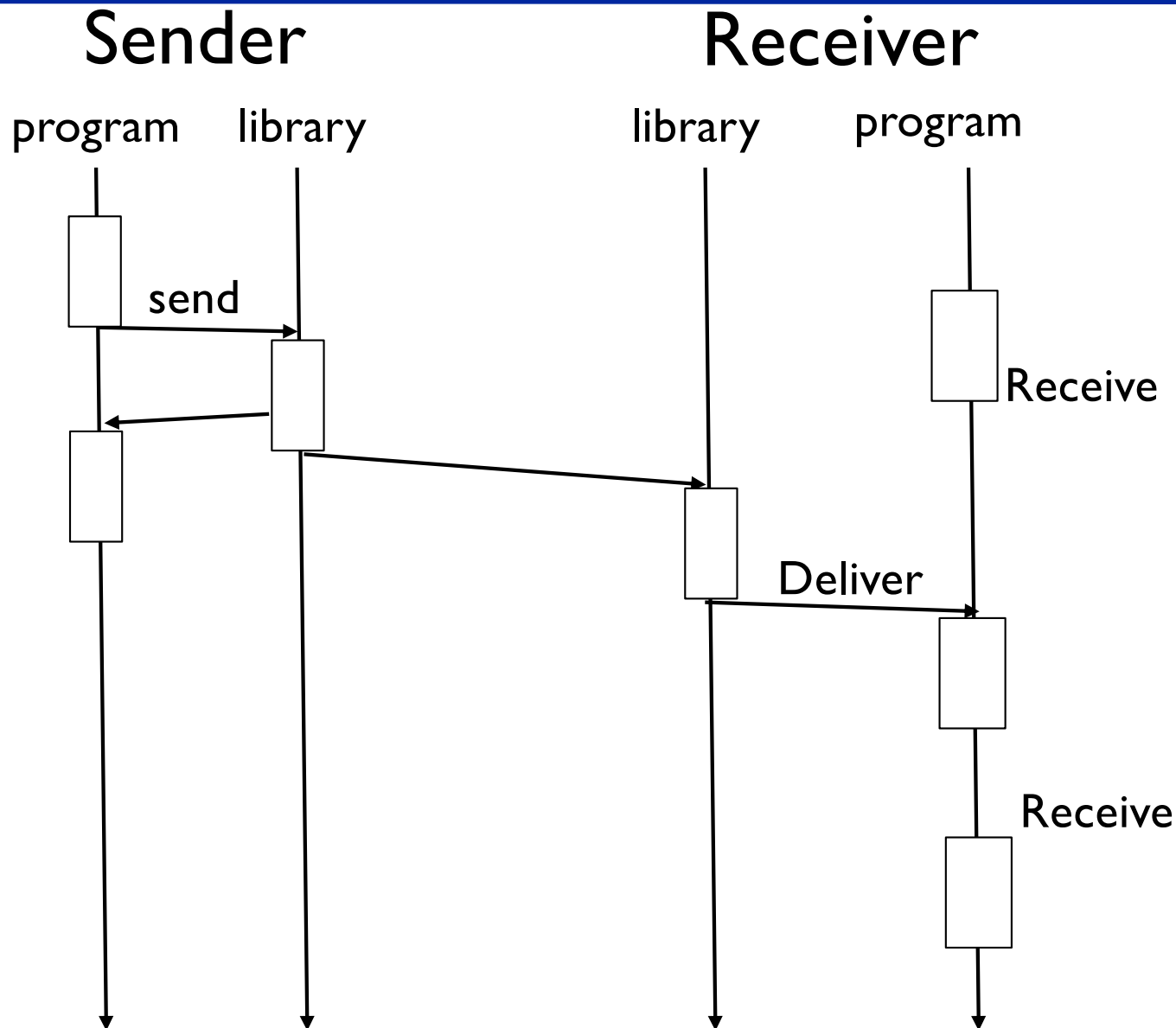
Inter-Process Communication



Blocking receive / non-blocking send



Delivery of a message



Non-blocking send blocking receive

❑ Non-blocking send

- ☆ send methods returns and sending process is allowed to proceed as soon as the underlying communication layer has received the message and committed to process it

❑ Blocking receive

- ☆ Receive method blocks until there is a message in the message buffer
- ☆ Multi-threaded environment:
 - one receiving listener thread that has loop with blocking receive
 - other threads do other work...
- ☆ *Default for the first two programming assignments*

Multi-threading

□ Receiver

Thread 1:

```
While () {  
    (m, s) = receive();  
    copy (m,s) and put into queue  
}
```

Worker Threads:

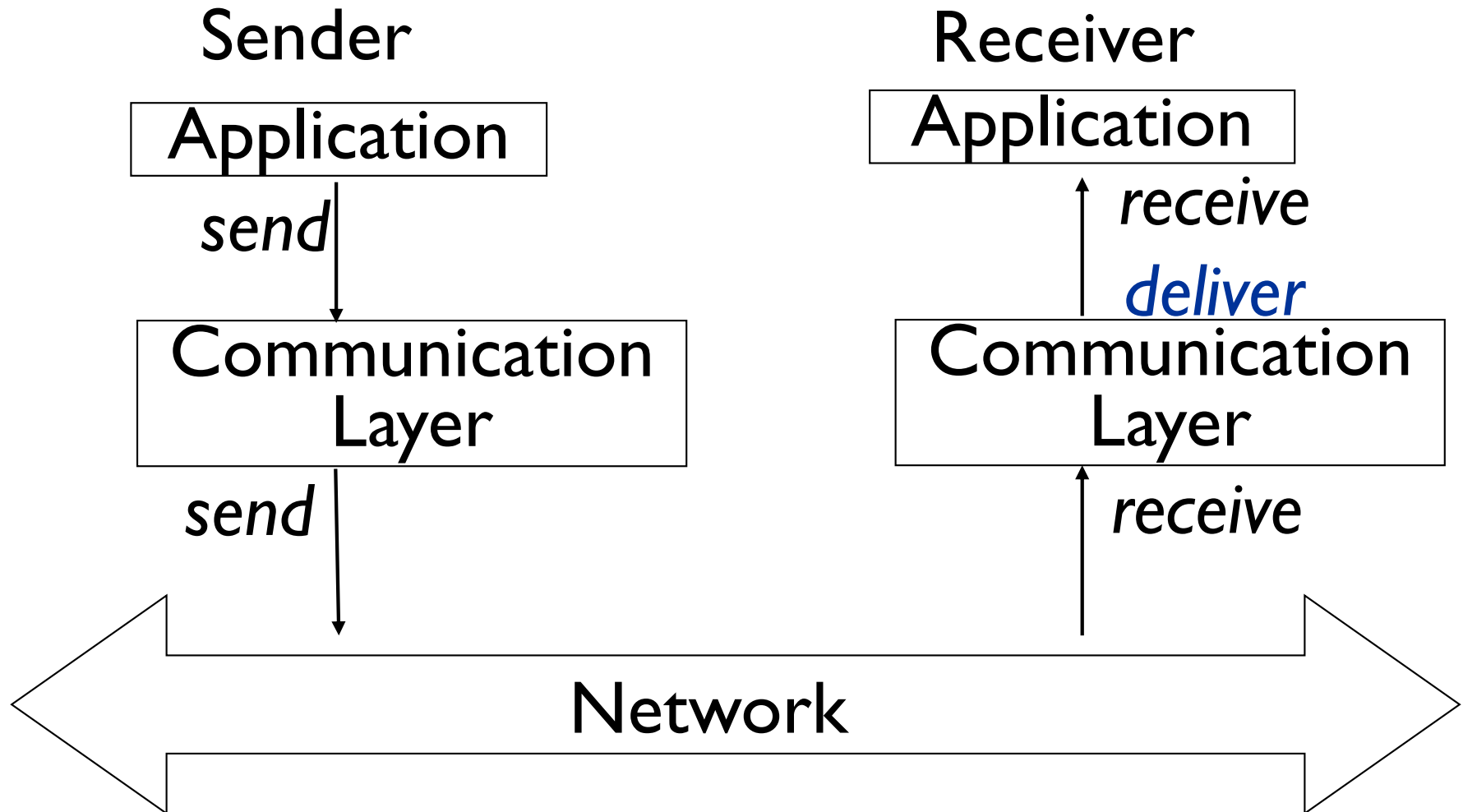
```
While () {  
    get message from queue;  
    process  
}
```

Alternative to Blocking receive

□ Message Handler

- ☆the application process provides the communication layer with a handler routine
- ☆the communication layer calls this routine upon message delivery
- ☆*Explored in the last programming assignment*

Basic Architecture



Source and Destination Addresses

- ❑ Abstract: process P

- ❑ Internet

 - ★ address (=host) + port (location dependent)

 - ★ Port is a message destination within a computer,

- ❑ Object (location independent)

 - ★ location of object is determined at runtime

- ❑ Service (location independent);

 - ★ Service name is translated at runtime to server
location

Performance

❑ Network Latency

- ☆ Delay between the start of transmission of a message over the network and the beginning of its receipt by receiving process
 - Time for the first bit to be transmitted through the network

❑ Process Latency

- ☆ Network Latency +
- ☆ Processing time at sender/receiver (communication layers)
 - Includes marshalling/unmarshalling
- ☆ Other delays due to load at sender/receiver, network

Performance

❑ Data transfer rate / bandwidth

★ Speed at which data can be transferred
between 2 computers

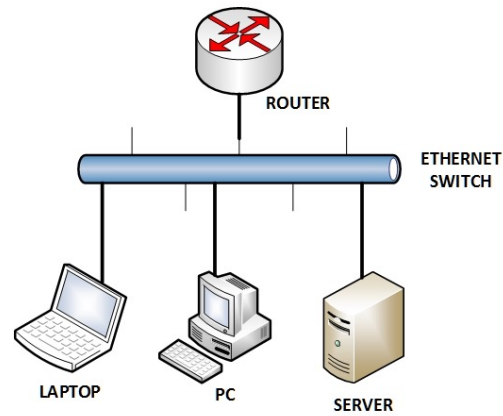
★ Bits/sec

❑ Jitter:

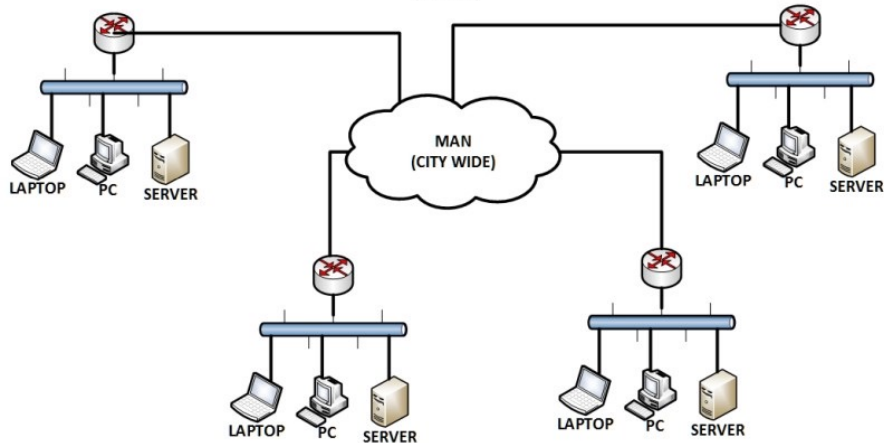
★ Variation in time taken to deliver a series of messages
(often bursty delivery)

Network Types

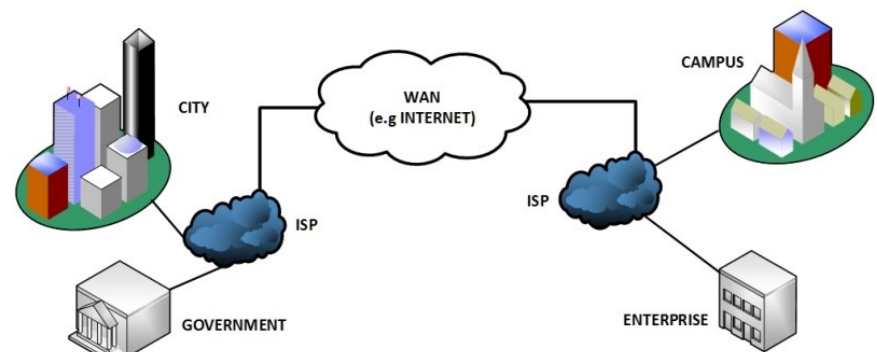
LOCAL AREA NETWORK
(LAN)



METROPOLITAN AREA
NETWORK (MAN)



WIDE AREA NETWORK
(WAN)

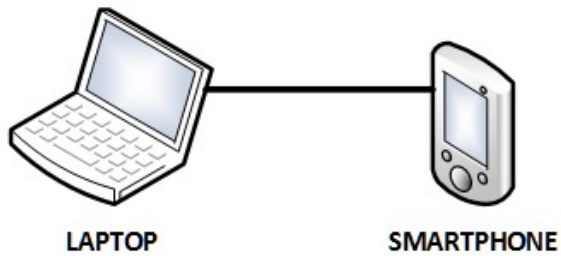


Network Types

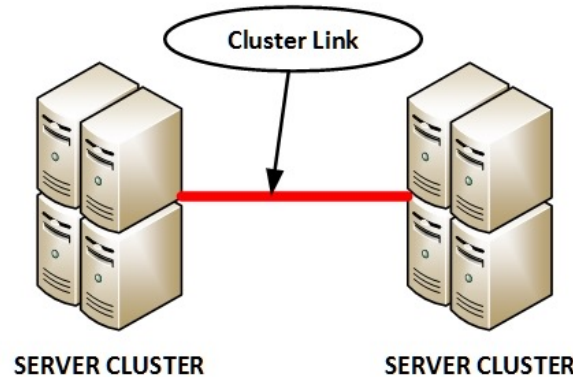
	<i>Range</i>	<i>Bandwidth (GBPS)/ Throughput</i>	<i>Latency (ms)</i>
LAN	1-2 kms	0.1-800	0.1-10
MAN (backbone)	2-50 kms	1-100	1-10
WAN/Internet	worldwide	0.0001-1	100-500

More network types

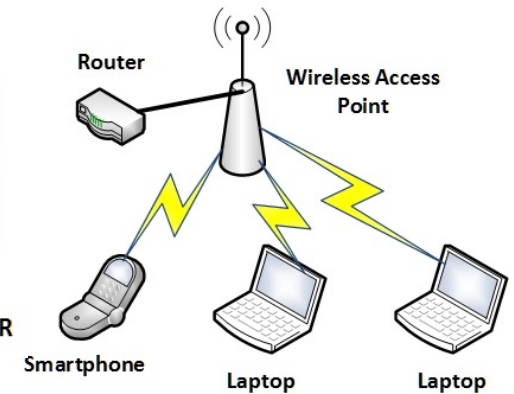
PERSONAL AREA NETWORK (PAN)



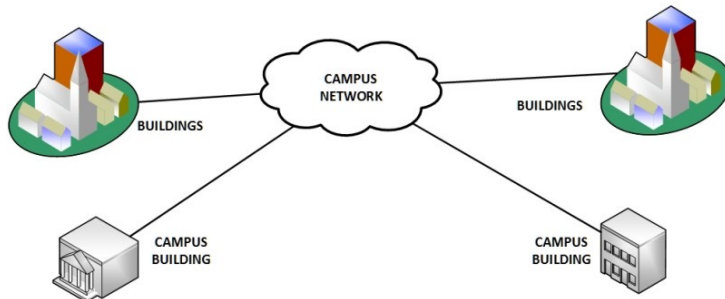
SYSTEM AREA NETWORK



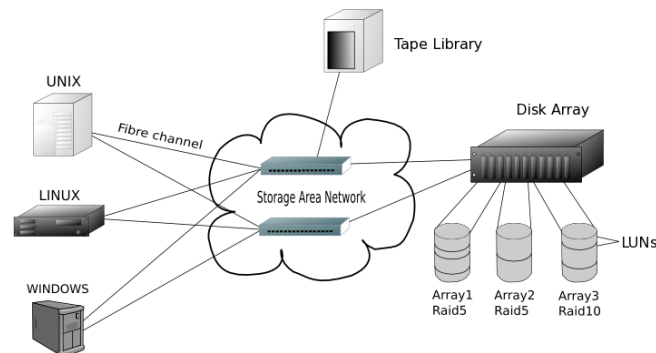
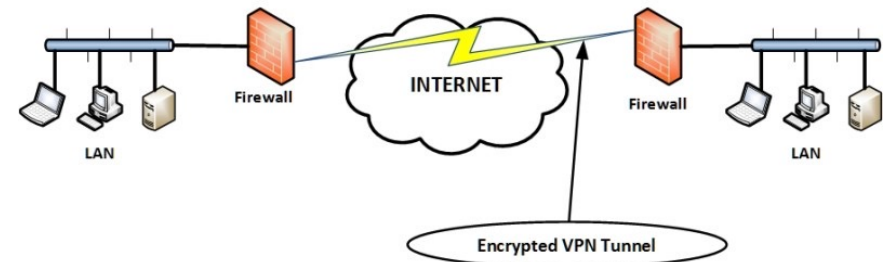
WIRELESS LOCAL AREA NETWORK (W-LAN)



CAMPUS AREA NETWORK (CAN)



VIRTUAL PRIVATE NETWORK (VPN)



Relative Bandwidth and Latency

- ❑ W-LAN / PLAN compared to LAN
- ❑ Campus network comparable to?
- ❑ System area network compared to general LAN
- ❑ SAN compared to general LAN
- ❑ VPN compared to the rest

Synchronous vs. asynchronous

❑ Synchronous distributed system

- ☆ each message transmits within a known bounded time
- ☆ The time to execute each step of a process has known lower and upper bounds
- ☆ Each process has a local clock whose drift rate from real time has a known bound

❑ Asynchronous distributed system as soon as any of the following holds

- ☆ Message may need an arbitrary time to be transmitted
- ☆ Each step of a process can take an arbitrary time
- ☆ Clocks drift rates are arbitrary

❑ In this course: asynchronous model if not stated otherwise

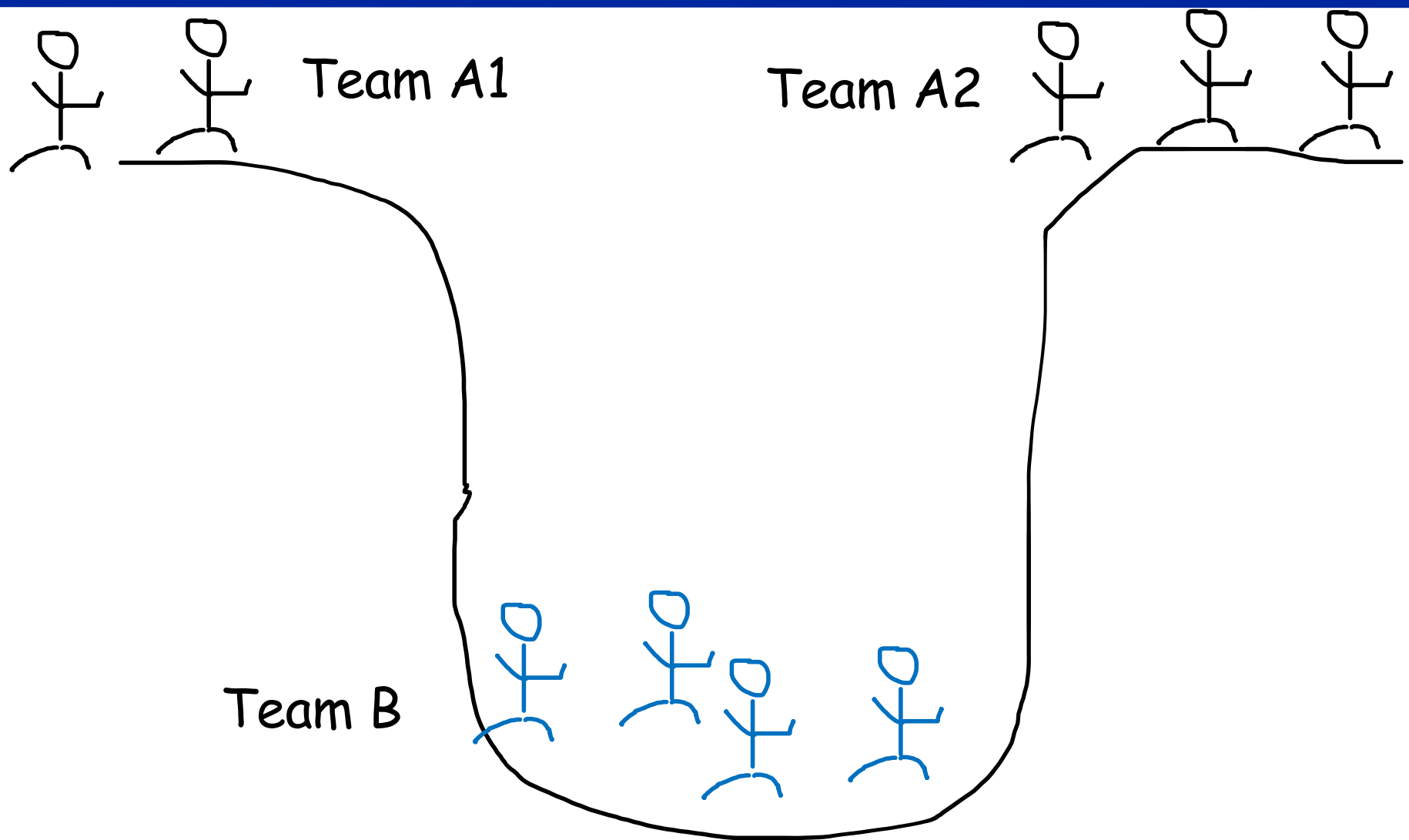
(A)synchronous Agreement

- ☆ Two teams A and B
- ☆ A split into two groups A1 and A2 on top of two mountains. Team B in the valley;
- ☆ A1 and A2 can exchange messengers

☆ **synchronous:**

- ☆ Each messenger needs at least *min* at most *max* time units
- ☆ (Clocks of A1 and A2 are off by at most *t* time units)

☆ **asynchronous but reliable:** messengers will *eventually* arrive but there is no time limit on how long the messenger needs



(A)Synchronous Agreement

□ Problem P1:

- ☆ Team A wants to "overrun" team B but to be successful, larger group has to start moving down first.
- ☆ How to determine **who starts?**

(A)Synchronous Agreement

□ Problem P2:

- ☆ Assume both teams know that A2 should start.
- ☆ To be successful, both groups A1 and A2 have to move down the mountains at nearly the same time (with at most X time units time difference)
- ☆ How to agree **when** to start moving??.
 - **Asynchronous model:**
 - **Synchronous model:**
 - ▲ Each messenger needs at least *min* at most *max* time units

Synchronous

Possible failures

❑ *Omission failures:*

☆ *process omission failure:*

- *fail-stop:*

- ▲ Process halts and remains halted;

- ▲ Others can detect this state

- *crash:*

- ▲ Process halts and remains halted;

- ▲ other processes may not be able to detect this

☆ *communication omission failures*

- receiving process does not receive message

- typically because of

- ▲ buffer overflow “somewhere”: individual **message loss**

- ▲ network partition

Possible failures II

❑ Process failures vs. network partitions:

☆ In general, it is impossible to distinguish between a process failure and a communication failure.

❑ *Arbitrary failures (byzantine failures)*

☆ processes might not follow agreed protocol

☆ messages might be corrupted

- can be solved by checksums

- **We will look at this in the context of bitcoin**

Failures and Point-to-point messages

❑ Process failures

★ If sender or receiver fails

- message exchange is no more possible

❑ Communication failure

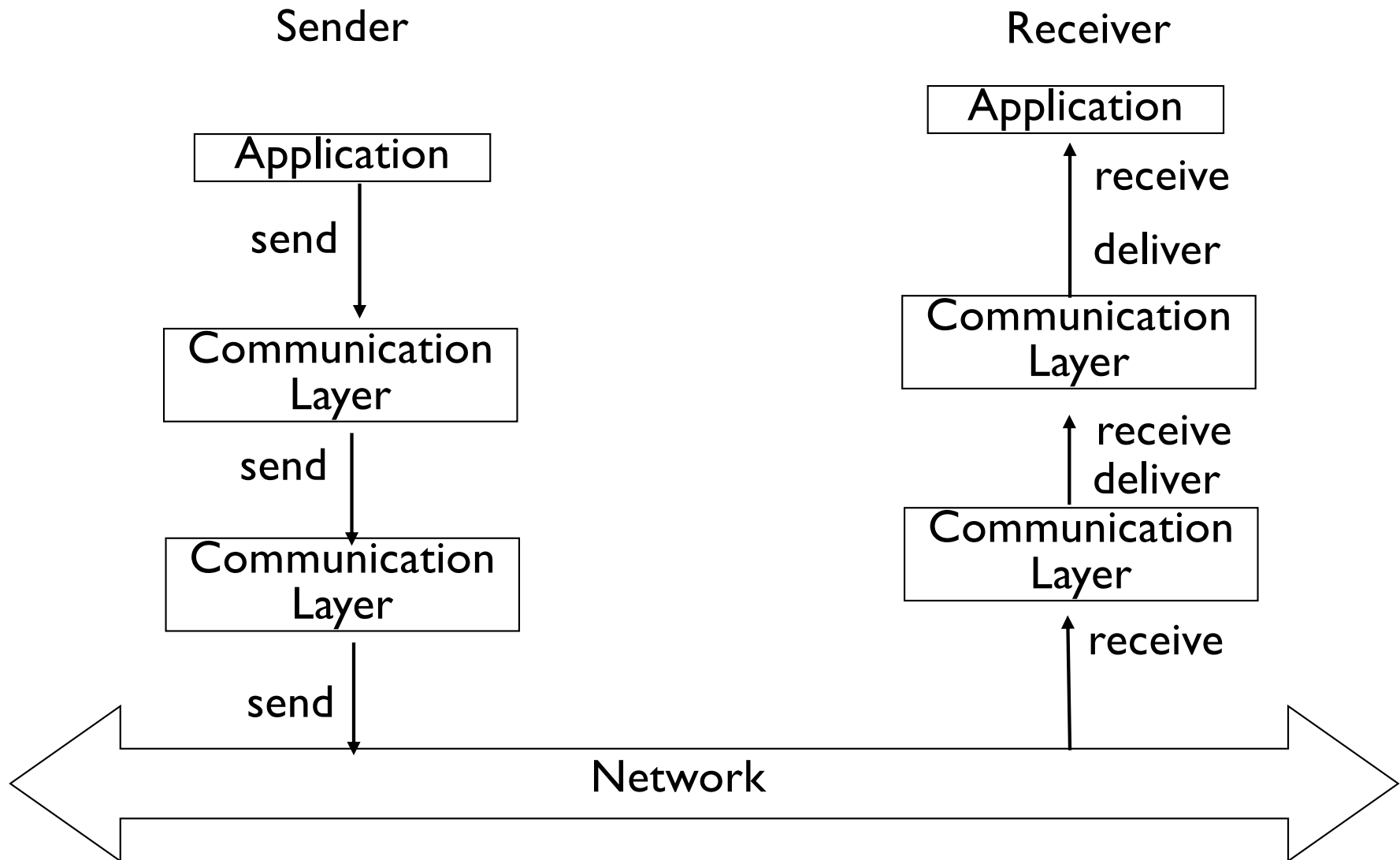
★ Network failure

- Message exchange no more possible until repaired

★ Individual message lost

- Something can be done

Layered Architecture



Reliability

- ❑ Assumption: no process failures, no network partition; messages might get lost
- ❑ Non-reliable delivery:
 - ★ The communication primitive only provides a best effort. That is, when no failures occur, the message will arrive. But in case of failures, it might not arrive, be duplicated, etc
 - ★ message identity typically provided
- ❑ Reliable delivery:
 - ★ *Validity*: any message sent is eventually delivered to the application (unless the application fails)
 - ★ *Integrity*: the message received is identical to the one sent, and no messages are delivered twice

Example

❑ Assume that

- ★ Processes never fail

- ★ No network partitions

- ★ Network provides unreliable `n_send(q, message)` / `n_receive()`

 - Messages can get lost

❑ Reliable communication module

- ★ Implement reliable send/receive using above unreliable `n_send/n_receive` pair

- ★ Basic ideas?

Reliability

- ★ TCP/UDP – IP protocols

- ★ TCP provides reliable message delivery despite *message loss*

- ★ UDP provides best-effort

- Careful !!!

- ★ You must specify the failures that your protocol can handle

- ★ TCP: individual message loss but not network partitions, node crashed or byzantine failures

- ★ The protocol must work correctly if the specified failures occur

- ★ The protocol may not work if unspecified failures occur